University of Heidelberg

Faculty of Mathematics and Computer Science

Institute of Computer Science

Research Group: Data Analysis and Modeling in Medicine

Bachelor's Thesis in Computer Science

Data-driven camera calibration for light field camera systems

Name: Youssef Daoudi El Boukhrissi
Student ID: 4277022
Supervisor: Prof. Dr. Jürgen Hesser
Submission date: 28 February 2026

Abstract

Contents

1. Introduction **1**
- 1.1. Motivation . . . 1
- 1.2. Challenges and Problem Statement . . . 1
- 1.3. Research Questions . . . 1
- 1.4. Structure of the Thesis . . . 1

2. Theoretical Background **2**
- 2.1. Pinhole Camera Model . . . 2
- 2.2. Lens Distortion . . . 3
  - 2.2.1. Radial Distortion . . . 3
  - 2.2.2. Tangential Distortion . . . 4
- 2.3. Camera Calibration . . . 5
  - 2.3.1. Target-Based Camera Calibration . . . 5
  - 2.3.2. Calibration Parameters and Error Metrics . . . 6
  - 2.3.3. Bundle Adjustment . . . 7
- 2.4. Light Field Camera Systems . . . 8
  - 2.4.1. Multi-Camera Light Field Systems . . . 8
  - 2.4.2. Micro-Lens-Array-Based Light Field Cameras . . . 9
- 2.5. State of the Art in Light Field Camera Calibration . . . 9

3. Methods and Materials **11**
- 3.1. Methods . . . 11
  - 3.1.1. Overall approach . . . 11
  - 3.1.2. Use Case Description . . . 11
  - 3.1.3. Requirements Analysis . . . 13
  - 3.1.4. UML Class Diagram . . . 15
  - 3.1.5. Health Monitoring and Decision Metrics . . . 16
  - 3.1.6. State Diagram . . . 17
- 3.2. Materials . . . 18
  - 3.2.1. Runtime Environment and Toolchain . . . 18
  - 3.2.2. Synthetic Data Generation with Blender . . . 18
  - 3.2.3. Image Sets and Ground Truth Definition . . . 20

4. Results and Evaluation **21**
- 4.1. Results of Initial Calibration . . . 21
  - 4.1.1. Intrinsic Accuracy . . . 21
  - 4.1.2. Extrinsic Accuracy . . . 21
  - 4.1.3. Reprojection and RMS Error . . . 21
- 4.2. Results of LiFCal-Based Recalibration . . . 21
  - 4.2.1. Reprojection Error after LiFCal . . . 21
  - 4.2.2. Health Score Evolution . . . 21

5. Discussion of the Results **22**
- 5.1. Interpretation of Initial Calibration Results . . . 22
- 5.2. Limitations of LiFCal in the Given Setup . . . 22
- 5.3. Comparison between Initial Calibration and LiFCal . . . 22
- 5.4. Comparison to state of the Art . . . 22

5.5. Potential Extensions and Improvements 22

6. Conclusion and Outlook **23**
6.1. Conclusion 23
6.2. Outlook 23

A. Appendix **23**

List of Figures

1. Illustration of the pinhole camera model. Source: [19]. 2
2. Comparison of different types of radial distortion. Source: [21]. 3
3. Visualization of tangential distortion. Source: [19], [18]. 4
4. Example of a checkerboard and detected corner points. Source: [14]. . . . 5
5. Visualization of reprojection error. Source: [2]. 6
6. Visual representation of bundle adjustment. Source: [15]. 7
7. Representation of a multi-camera system. Source: [29]. 8
8. Structure of a micro-lens-array-based light field camera. Source: [8]. . . . 9
9. UML Class Diagram 15
10. State Diagram 17
11. Multi-camera array setup in Blender. 18
12. Example of a Blender scene for the initial calibration with a checkerboard. 19
13. Example of a Blender scene for LiFCal recalibration without a calibration target. 19

1. Introduction

1.1. Motivation

1.2. Challenges and Problem Statement

1.3. Research Questions

The following research questions guide this thesis:

- **RQ1:** Can the target-free calibration method LiFCal be used to automatically calibrate a multi-camera light field system without human interaction or the use of a checkerboard?
- **RQ2:** How well does LiFCal perform when calibrating a multi-camera light field system, and can it replace a classical checkerboard-based calibration?
- **RQ3:** Under which conditions and requirements can an automatic, target-free recalibration of a multi-camera light field system be successfully performed?

1.4. Structure of the Thesis

2. Theoretical Background

This chapter introduced the theoretical foundations required for this thesis. It covered the pinhole camera model, including intrinsic and extrinsic parameters as well as lens distortion, as the basis for geometric camera modeling. Building on this, common camera calibration methods and error metrics were presented, including target-based calibration and bundle adjustment. The chapter also introduced light field camera systems, their main system types, and the state of the art in their calibration. Together, these concepts provide the necessary background for the calibration approach and experiments presented in the following chapters.

2.1. Pinhole Camera Model

Cameras are the primary devices used to capture visual information from the real world into an image. To process and analyze this information computationally, it is necessary to describe the imaging process using a camera model [17].

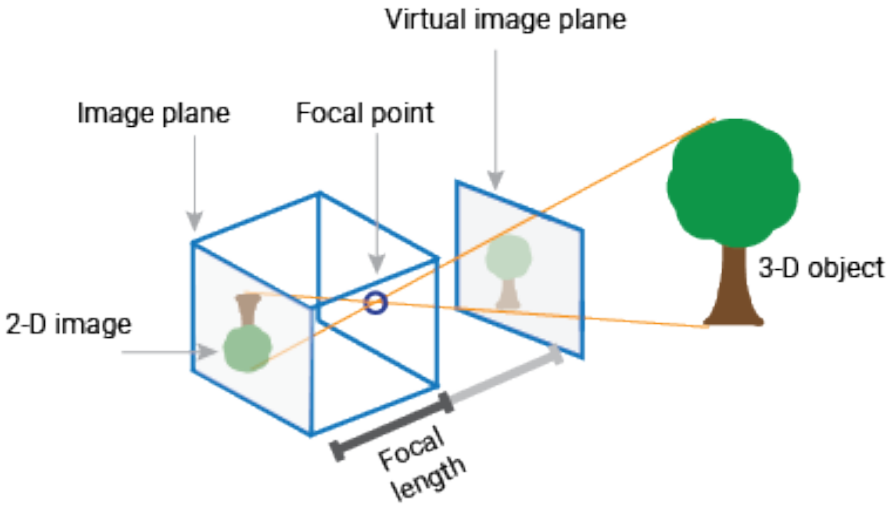

Figure 1: Illustration of the pinhole camera model. Source: [19].

The pinhole camera model describes a simple imaging system in which light rays from a 3D scene pass through a small aperture and are projected through the focal point onto a 2D image plane [17]. The aperture limits the incoming light such that each point in the scene is mapped to a single point on the image plane. In this model, the projected image is inverted due to the geometry of the projection, as shown in Figure 1.

To define the mapping from 3D world coordinates to 2D image coordinates, the model is mathematically described using two types of parameters [19].

- **Intrinsic Parameters:**

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

  where (f_x, f_y) are the focal lengths in pixel units along the x and y axes, s is the skew coefficient between the axes, and (c_x, c_y) is the principal point (optical center) in pixel coordinates. K is also known as the camera matrix.

- **Extrinsic Parameters:**

$$[R \mid t] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}$$

  where R is a 3×3 rotation matrix representing the orientation of the camera with respect to the world coordinate system, and t is a 3×1 translation vector representing the position of the camera center in world coordinates.

These parameters form the basis of camera calibration, which aims to estimate them accurately. However in practice, cameras use lenses to focus light onto the image sensor, which introduces distortions not accounted for in the idealized pinhole model.

2.2. Lens Distortion

The camera matrix alone is not sufficient to model real imaging systems, as the ideal pinhole camera model assumes a lens-free projection. In practice, however, camera lenses introduce various optical distortions that must be taken into account to accurately describe the image formation process [19].

Lens distortion is defined as an optical aberration that causes straight lines in the real world to appear curved in the captured image [7]. Since distortion alters the geometric relationship between 3D world points and their 2D image projections, it has a direct impact on the accuracy of camera calibration. If lens distortion is not properly modeled, the estimated intrinsic and extrinsic parameters can become biased, leading to unreliable calibration results [23].

In most camera calibration models, lens distortion is represented using a small set of parametric functions. The two most common types of distortion considered in practice are radial distortion and tangential distortion.

2.2.1. Radial Distortion

Radial distortion is caused by the spherical shape of camera lenses, which leads to different refraction of light from the image center to the outer regions. This causes straight lines in the image to appear curved, resulting in barrel or pincushion distortion [6].

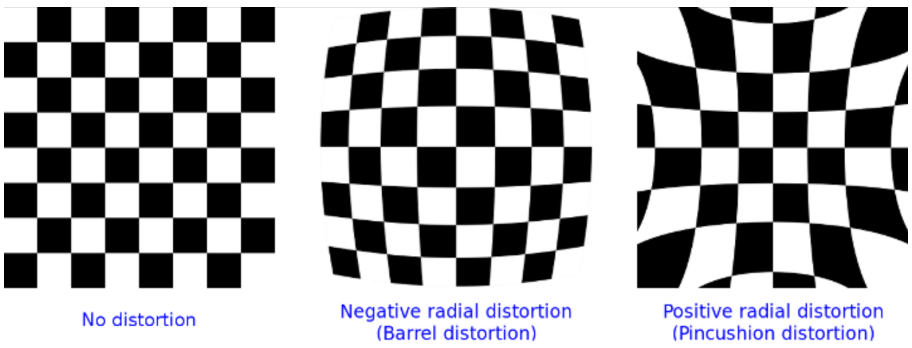

Figure 2: Comparison of different types of radial distortion. Source: [21].

The formal definition of radial distortion is given as follows [19]:

$$x_{distorted} = x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) \tag{1}$$

$$y_{distorted} = y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) \tag{2}$$

where x and y denote the undistorted normalized image coordinates in the camera coordinate system. The radius r represents the distance of a point from the image center and is defined as:

$$r^2 = x^2 + y^2 \tag{3}$$

The coefficients k_1, k_2, and k_3 in (1) and (2) control the strength of radial lens distortion. In practice, the first coefficient k_1 has the dominant influence on the distortion type: negative values of k_1 typically result in barrel distortion, while positive values lead to pincushion distortion.

2.2.2. Tangential Distortion

Tangential distortion occurs when the image sensor and the camera lens are not perfectly aligned or parallel to each other [19]. As a result, image points are displaced tangentially with respect to the optical axis. Figure 3 illustrates the geometric cause of tangential distortion and its effect on the captured image.

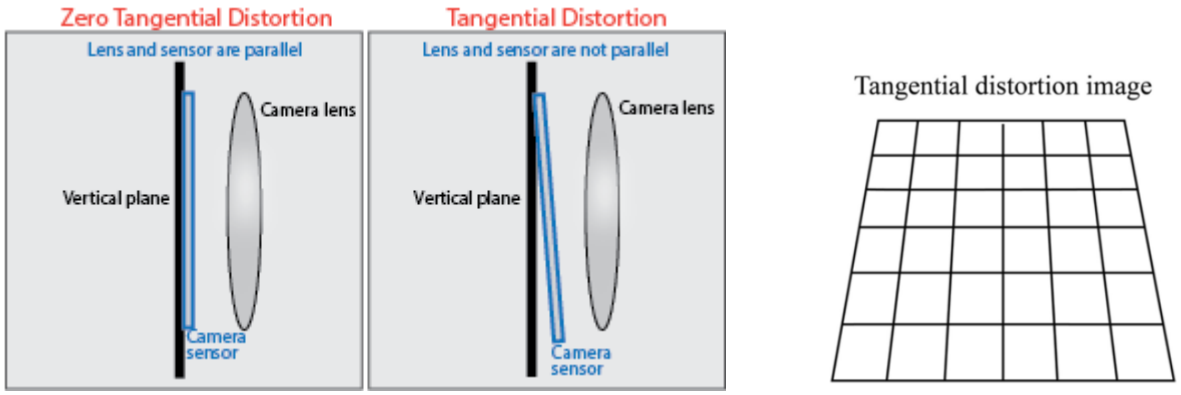

Figure 3: Visualization of tangential distortion. Source: [19], [18].

The formal definition of tangential distortion is given by the following equations [19]:

$$x_{\text{distorted}} = x + \left(2p_1 xy + p_2\left(r^2 + 2x^2\right)\right) \tag{4}$$

$$y_{\text{distorted}} = y + \left(p_1\left(r^2 + 2y^2\right) + 2p_2 xy\right) \tag{5}$$

Here, x and y denote the undistorted normalized image coordinates in the camera coordinate system. The radius r represents the distance of a point from the image center and is defined as:

$$r^2 = x^2 + y^2 \tag{6}$$

The coefficients p_1 and p_2 describe how strong the tangential lens distortion is. This type of distortion is caused by mechanical misalignment of the lens elements relative to the image sensor, resulting in a displacement of image points. Although tangential distortion is typically less pronounced than radial distortion, it can still negatively affect the camera calibration accuracy if not properly modeled.

2.3. Camera Calibration

The goal of camera calibration, also known as camera resectioning, is to estimate the parameters of a camera model. These parameters include the intrinsic parameters, the extrinsic parameters, and the lens distortion coefficients [19]. Knowing these parameters makes it possible to correct distortions in the recorded images and to perform metric measurements in the scene, for example to estimate object sizes or distances from image data.

Camera calibration is required for many tasks in 3D computer vision and photometry. Applications such as depth estimation, 3D reconstruction, or pose estimation rely on a precise mapping from 3D world coordinates to 2D image coordinates [25]. If the calibration is inaccurate, errors can propagate through the processing pipeline and reduce the quality of the final results.

In practice, cameras are often calibrated using targets with known geometric properties. Planar targets, such as checkerboard patterns, are widely used because they are easy to handle and lead to reliable results [32]. Several images of the target are captured from different viewpoints and orientations. Based on these images, the camera parameters can be estimated with relatively small calibration errors. For this reason, target-based calibration methods are commonly used in many computer vision applications.

2.3.1. Target-Based Camera Calibration

There are various approaches for target-based camera calibration. One of the most common methods uses a checkerboard pattern, as mentioned before. The checkerboard consists of a regular grid of black and white squares with known dimensions, as shown in Figure 4.

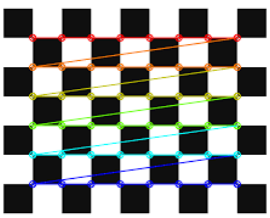

Figure 4: Example of a checkerboard and detected corner points. Source: [14].

During the calibration process, control points are extracted from the images. In the case of a checkerboard, these control points correspond to the corner points of the squares [25]. The positions of these corners are known in the coordinate system of the calibration target. By capturing multiple images of the checkerboard from different angles and distances, a set of 2D image points can be obtained that correspond to the known 3D points of the target. Based on these point correspondences, an optimization problem can be formulated to estimate the intrinsic and extrinsic camera parameters as well as the lens distortion coefficients [32].

This calibration approach is implemented in widely used computer vision libraries such as OpenCV [20]. It can be applied to single-camera setups and multi-camera systems. Besides checkerboards, other types of calibration targets exist, for example circular dot patterns [25]. In some cases, these patterns can achieve higher accuracy, especially when the image quality is low. However, they also require physical targets and a manual calibration setup.

The main limitations of target-based calibration methods are the required manual placement of the target and the need to capture images with different poses. In addition, the calibration accuracy depends on factors such as the quality of the corner detection, the number of images used, and the diversity of viewpoints during data acquisition [32]. Despite these limitations, target-based calibration remains a standard method in many computer vision applications due to its simplicity and reliable performance.

2.3.2. Calibration Parameters and Error Metrics

To evaluate the quality of a camera calibration, different error metrics are used. These metrics describe how well the estimated camera parameters fit the observed image data. In practice, this is done by comparing detected image points with points that are projected using the calibrated camera model. The most common metric is the reprojection error [19].

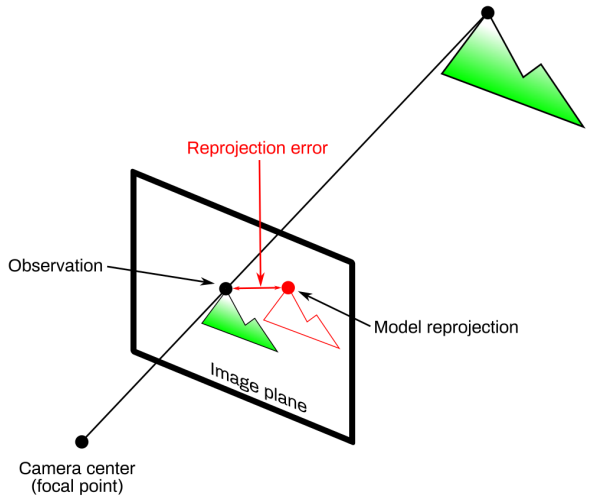

Figure 5: Visualization of reprojection error. Source: [2].

After a calibration, known 3D points are projected back onto the image plane. The reprojection error measures the distance between the observed image points and the

corresponding projected points. It is usually computed as the average Euclidean distance and is given in pixel units [2]. It can be defined as

$$e = \frac{1}{N} \sum_{i=0}^{N-1} \|\mathbf{p}_i - \mathbf{q}_i\|_2. \tag{7}$$

Here, $\mathbf{p}_i$ represents the observed feature points in the image, while $\mathbf{q}_i$ denotes the projected points obtained from the calibrated camera model. In general, a smaller reprojection error indicates a better calibration. What is considered a good value depends on the application, but for accurate calibrations the reprojection error is often between 0.1 and 0.5 pixels [12].

Another commonly used measure is the root mean square (RMS) error [11]. It summarizes the reprojection error over all calibration points and is computed as:

$$\epsilon_{\text{res}} = \left(\frac{1}{2n} \sum_{i=1}^{n} \left\| \mathbf{p}'_i - \mathbf{q}'_i \right\|_2^2 \right)^{1/2} \tag{8}$$

The RMS error provides a single value that describes the overall calibration accuracy and is often used to compare different calibration results.

2.3.3. Bundle Adjustment

Bundle adjustment is a method for improving a 3D reconstruction by refining both the positions of 3D points and the camera parameters at the same time [27]. The name comes from the "bundles" of light rays from the 3D points to the cameras, which are adjusted together to make the reconstruction more accurate. The parameters that are optimized together usually include the 3D coordinates of the points, the extrinsic parameters, and sometimes the intrinsic parameters [4].

This joint optimization improves calibration accuracy because it ensures that all cameras and points are consistent with each other. As a result, projection errors are reduced and small misalignments are corrected that would remain if cameras or points were adjusted separately [27]. This idea is illustrated in Figure 6.

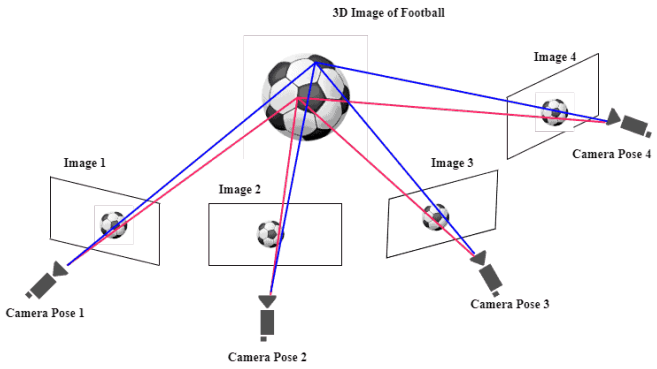

Figure 6: Visual representation of bundle adjustment. Source: [15].

Mathematically, bundle adjustment is formulated as an optimization problem that minimizes the reprojection error. This optimization is usually solved using nonlinear least squares methods and takes advantage of the fact that the problem structure is largely sparse [4].

2.4. Light Field Camera Systems

A standard camera captures a 2D projection of the 3D world. Light field cameras, on the other hand, have the ability to capture more information. They record not only the intensity of light rays but also their direction [10]. This property allows the creation of accurate depth maps and refocus the image after it has been taken [8]. Therefore light field cameras are growing in importance for applications in computer vision, robotics, augmented reality, and other applications.

The four-dimensional nature of light fields means that their processing currently needs relatively high computational resources [10]. However, the evolution of hardware and algorithms is making light field cameras more practical for real-world applications.

There are mainly two different types of light field camera systems: multi-camera arrays and micro-lens-array-based cameras. Each type has its own advantages and disadvantages, which will be discussed in the following sections.

2.4.1. Multi-Camera Light Field Systems

Multi-camera arrays are one way to capture light fields. This type of system is structured by arranging multiple individual cameras in a geometric configuration [29]. Each camera captures a different viewpoint of the scene, enabling the system to record a 4D light field. In practice, there are different approaches to arranging the cameras. For example, in [26], an array of 128 cameras in a square grid is used, while in other works cross-shaped structures with fewer cameras are employed [29], as shown in Figure 7.

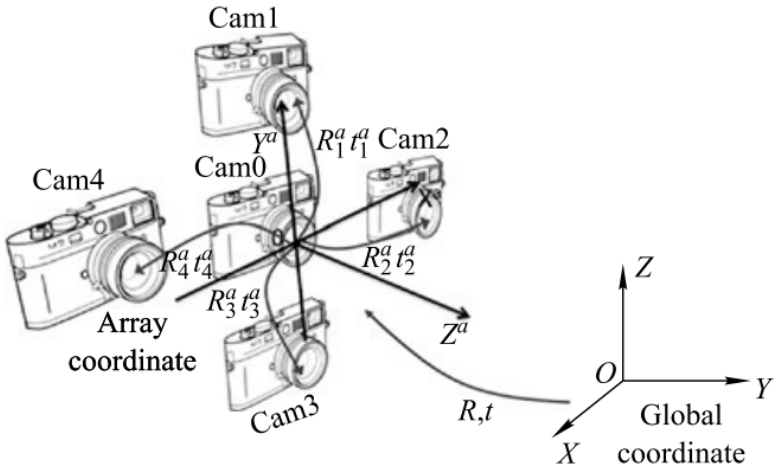

Figure 7: Representation of a multi-camera system. Source: [29].

Besides the advantages of a multi-camera setup, such as improved depth accuracy and refocusing capabilities, there are also challenges. The calibration of such systems is more

complex and requires higher computational effort due to the larger number of cameras involved [29].

It is also essential to keep the cameras properly aligned and synchronized. Even small errors in alignment or timing can cause artifacts or undesired results in the final image. Therefore, multi-camera light field systems require a very precise setup and calibration to work as intended.

2.4.2. Micro-Lens-Array-Based Light Field Cameras

The second type of light field camera is based on a micro-lens array (MLA). In this case instead of having different cameras, the system uses only one sensor combined with an array of small lensen placed between the main lens and the image sensor [8]. The MLA allows the camera to capture also the 4D light field information.

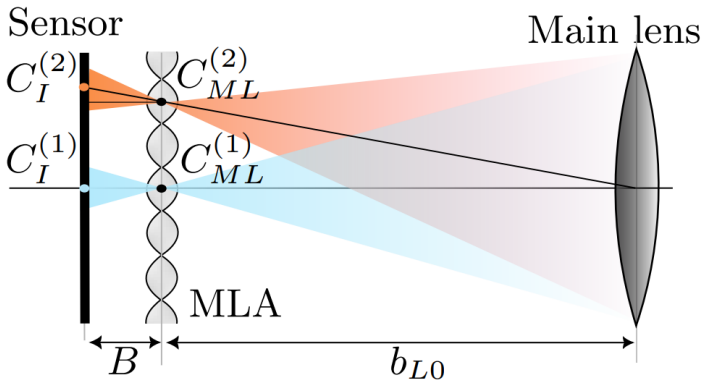

Figure 8: Structure of a micro-lens-array-based light field camera. Source: [8].

As illustrated in Figure 8, each lens in the MLA focuses light from different directions onto different regions of the sensor. This way, the camera is also able to cpature the same information as a multi-camera array, but with a more compact design. Because of the smaller size and lower cost, MLA-based light field camera are mostly used in consumer applications [16]. However, microlens-array cameras have to compromise between spatial and angular resolution, while multi-camera systems capture full-resolution images with sharper results and more accurate depth information.

For MLA-based light field cameras, the pattern-based calibration method provides satisfactory results in terms of reprojection error. In practice, the overall calibration quality strongly depends on the calibration target, particularly its flatness [1].

2.5. State of the Art in Light Field Camera Calibration

Accurate calibration is a central requirement for light field camera systems, as even small geometric errors directly affect depth estimation and view consistency. In camera arrays, where many individual views must agree, calibration errors accumulate and lead to visible artifacts in depth maps and reconstructed images.

Early work on light field calibration mainly relies on pattern-based approaches. Dima et al. systematically evaluate several multi-camera calibration methods using ground truth

data and show that calibration accuracy strongly depends on the array geometry and the physical setup of the calibration target [5]. In particular, small deviations such as bending or uneven surfaces of the calibration board already lead to increased reprojection errors across the array. Yang et al. demonstrate that using a simple checkerboard target can still yield stable calibration results for camera arrays while reducing setup complexity [30]. However, the accuracy of these methods remains closely tied to target quality and careful placement.

To overcome the limitations of target-based calibration, several self-calibration approaches have been proposed. Zhang et al. introduce a fixed relative pose prior to stabilize self-calibration in camera arrays [31]. By constraining relative camera poses, their method reduces pose drift in situations where image features are sparse or unevenly distributed. Zhou et al. address low-texture sports scenes and use temporal fusion across multiple frames to estimate calibration parameters [33]. These approaches reduce the need for calibration targets, but still depend on sufficient scene structure and a reasonable initial alignment.

Some methods focus on specific system layouts or downstream tasks. Ge et al. study a ring-shaped multi-camera system and use a transparent calibration target to support panoramic measurements [9]. Xie et al. show that accurate camera array calibration is essential for reliable depth map fusion, as calibration errors propagate directly into depth artifacts, especially near object boundaries [28].

The approach most relevant to this thesis is LiFCal by Fleith et al. [8]. Instead of using a physical calibration target, LiFCal moves the MLA-based camera through the scene and estimates calibration parameters via bundle adjustment driven by depth maps from multiple viewpoints. This shifts the calibration problem from controlled target acquisition to scene structure and camera motion, making the method particularly suitable for scenarios where calibration targets are impractical or unavailable.

Overall, existing work shows that calibration accuracy in light field systems remains sensitive to setup quality, motion, and scene structure. Larger camera arrays amplify small errors, and target-free methods require reliable depth estimates and sufficient camera motion to properly constrain the geometry.

3. Methods and Materials

This chapter introduces the methods and materials used in this thesis, covering both the design of the calibration framework and the technical setup used for its implementation and evaluation.

3.1. Methods

In this first section, the design and implementation of the calibration software are described. It describes the overall workflow, the use case scenario, and the specific requirements that guided the development process. The UML class diagram and state diagram illustrate the software architecture and behavior.

3.1.1. Overall approach

The goal of this thesis is to develop and evaluate an automatic calibration framework for multi-camera light field systems based on the target-free calibration method proposed by Fleith et al. [8]. As discussed in the state of the art, classical target-based calibration approaches require manual intervention and careful placement of calibration targets, which limits their applicability in long-term deployments or scenarios with restricted human access.

The proposed approach uses a reliable target-based calibration as an initial reference and ground truth. This initial calibration is performed using a checkerboard pattern and serves as a baseline for evaluating subsequent target-free recalibration steps. After this initial setup, the system captures a small set of images from the scene and applies LiFCal to automatically recalibrate the multi-camera system. To assess whether the target-free recalibration is acceptable, a health monitoring mechanism is introduced. This mechanism compares the calibration results obtained from LiFCal with the baseline calibration using defined decision metrics. If the recalibration achieves a calibration quality comparable to or better than the initial calibration, it is accepted and replaces the previous baseline. Otherwise, the system retains the existing calibration parameters.

This workflow is designed to enable periodic self-recalibration without manual intervention. A potential application scenario is a robotic system deployed in a remote or inaccessible environment, where mechanical shifts or environmental changes may degrade calibration quality over time. By regularly capturing image sets and applying the proposed recalibration process, the system can maintain accurate camera parameters throughout its operation.

To clearly describe the interaction between the individual components and calibration phases, a use case description is introduced in the following section.

3.1.2. Use Case Description

The following use case describes the intended workflow of the proposed automatic calibration framework for a cross-shaped multi-camera light field system consisting of a central camera and twelve surrounding cameras, arranged in four cardinal directions (North, East, South, and West). The workflow is structured into three consecutive phases: an

initial reference calibration, periodic target-free recalibration, and a health-based decision step.

Phase 1 — Initial Reference Calibration (once)

- **Target-based calibration (OpenCV):** A one-time checkerboard-based calibration is performed to estimate the intrinsic and extrinsic parameters for all cameras of the cross-shaped array.
- **Optional refinement:** An optional global refinement using bundle adjustment is applied to improve consistency across all camera parameters.
- **Baseline creation:** A global health score is computed for the initial calibration and stored as `baseline.json`. This baseline represents the known-good reference calibration for subsequent recalibration decisions.
- **Logging:** The complete initial calibration is stored as a timestamped JSON file and a summary entry is appended to `calibration.log`.

Phase 2 — Automated Recalibration Attempts (periodic scheduler)

- **Trigger:** An automated recalibration cycle is executed at fixed time intervals (e.g. every five minutes).
- **Conditional data preparation:** If LiFCal-compatible data (focus and depth directories) already exists and is complete, it is reused. Otherwise, the system generates the required inputs by running `run_imageset_creation.py`.
- **LiFCal recalibration run:** The script `run_LiFCal_calibration.py` executes LiFCal-based recalibration for all cameras in the array and produces per-camera parameter files as well as a combined multi-camera result.
- **Logging and traceability:** The combined recalibration result is stored as a timestamped JSON file and all relevant execution details are appended to `calibration.log`.

Phase 3 — Health Evaluation and Baseline Update Rule

- **Health score computation:** A single global health score in the range $[0, 100]$ is computed from the combined LiFCal calibration result.
- **Comparison against baseline:** The newly computed health score is compared to the score stored in the current baseline calibration.
- **Acceptance rule:** If the new health score is strictly higher than the baseline score, the baseline calibration is replaced by the new result. Otherwise, the existing baseline remains unchanged.
- **Continuous improvement loop:** The recalibration process is repeated periodically, allowing improved calibration results to update the baseline over time without manual intervention.

This structure allows the system to maintain accurate calibration parameters over time, adapting to changes in the environment or camera setup. The health-based decision mechanism ensures that only beneficial recalibrations are accepted, preserving the integrity of the calibration throughout the system's operation.

This use case serves as the conceptual basis for the requirements analysis and system design presented in the following sections.

3.1.3. Requirements Analysis

The requirements for the proposed automatic calibration framework are divided into functional and non-functional requirements. These requirements define the necessary features and qualities of the system to achieve the intended use case.

Functional Requirements

- **FR1 – Initial Reference Calibration (Target-based):**
  The system shall perform a one-time, checkerboard-based initial calibration using OpenCV to estimate intrinsic and extrinsic parameters for the cross-shaped camera array. This initial calibration shall be stored as the baseline calibration and serve as the reference state for all subsequent health evaluations and recalibration decisions.

- **FR2 – Parameter Export and Standardized Representation:**
  The system shall export all calibration results (initial calibration and LiFCal-based recalibration) in a standardized, machine-readable JSON format, including intrinsic parameters, extrinsic poses, reprojection error statistics, timestamps, and metadata.

- **FR3 – Periodic Calibration Health Monitoring:**
  The system shall periodically (at least once per configured scheduler interval) evaluate the calibration quality by computing a global health score.

- **FR4 – Health Score Computation (Global Scalar Metric):**
  The system shall compute a single scalar health score in the range $[0, 100]$ representing the overall calibration quality of the complete camera array. For the initial calibration, the health score shall be derived from reprojection error statistics. For LiFCal-based recalibration, the health score shall be derived from LiFCal-specific reprojection and consistency metrics extracted from the combined parameter set.

- **FR5 – Target-Free Online Recalibration (LiFCal-based):**
  The system shall execute a LiFCal-based target-free recalibration using automatically prepared focus images and virtual depth data whenever a recalibration cycle is triggered.

- **FR6 – Acceptance and Baseline Update Rule:**
  The system shall compare the health score of a newly computed calibration against the health score of the currently stored baseline calibration. The new calibration shall be accepted and stored as the new baseline only if its health score is strictly higher than the baseline score; otherwise, the system shall retain the previous baseline calibration.

- **FR7 – Automated Periodic Recalibration Scheduling:**
  The system shall support automated, periodic execution of the LiFCal-based recalibration process at a configurable time interval (minimum every 5 minutes).
- **FR8 – Logging and Traceability:**
  The system shall log every calibration and recalibration run, including at least timestamps, used image sets, computed health scores, baseline update decisions, and error states.
- **FR9 – Abort and Logging on Recalibration Failure:**
  The system shall abort the current recalibration cycle if a recalibration run fails. In such cases, the system shall retain the currently active baseline calibration and record a descriptive error message, including the failure reason, in the calibration log.

Non-Functional Requirements

- **NFR1 – Modularity:**
  The system shall be structured into modular components (initial calibration, LiFCal recalibration, health monitoring, scheduling, and logging) to support extensibility and maintainability.
- **NFR2 – Reproducibility:**
  All calibration results and health evaluations shall be reproducible using stored JSON files and logged metadata.
- **NFR3 – Robustness:**
  The calibration and monitoring pipeline shall tolerate moderate variations in scene content and illumination without system failure, even if recalibration quality temporarily degrades.
- **NFR4 – Automation:**
  The calibration framework shall operate fully automatically after initial setup, without requiring manual intervention during periodic monitoring and recalibration.
- **NFR5 – Transparency:**
  The system shall provide access to intermediate calibration metrics and logged decision outcomes to allow inspection and analysis of health score computations and acceptance decisions.

These requirements form the basis for the system design and implementation presented in the following sections. They were derived following the guidelines and best practices of IEEE Std 830–1998 [13] and directly support the intended use case of the proposed calibration framework. The functional requirements are reflected in the architectural decisions, UML diagrams, and implementation steps, while the non-functional requirements address relevant quality aspects such as robustness, automation, and maintainability.

The requirements specification is intentionally kept compact and focused. It is not intended to represent a complete industrial requirements document, but rather a research-oriented specification that is appropriate for the scope and objectives of this bachelor thesis.

3.1.4. UML Class Diagram

Based on the requirements analysis and the defined use case, a UML class diagram was designed to structure and illustrate the main components of the project. The diagram in Figure 9 shows the key classes, their attributes, methods, and relationships.

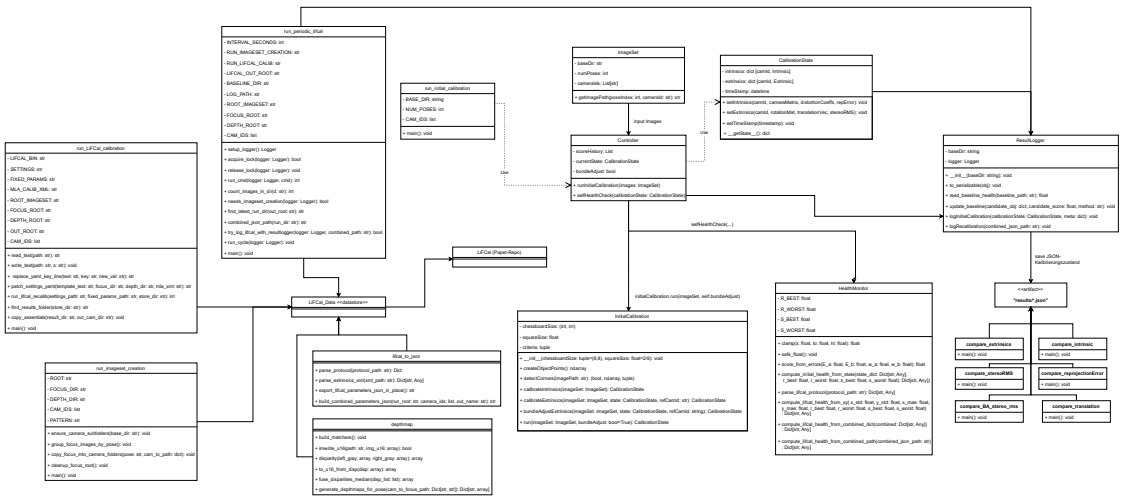

Figure 9: UML Class Diagram

The software architecture is based on a modular design, so that individual components can be developed, tested, and maintained independently. If future extensions or modifications are needed, the structure allows the easy integration of new features without major changes to the existing codebase.

The core class is the `controller`, which manages the overall calibration workflow. It coordinates the initial calibration, periodic recalibration attempts, health monitoring, and logging. The controller interacts with several classes that contain the specific functionalities.

The initial calibration and periodic recalibration are implemented in the `InitialCalibration` and `run_LiFCal_calibration` classes, respectively. To run these classes correctly, the `run_initial_calibration`, `run_periodic_lifcal`, and `run_imageset_creation` classes are responsible for preparing the required data and executing the calibration processes. In addition, the `depthmap` class creates, based on the image set, a depth map for each image, which is used for the LiFCal calibration. For health monitoring and decision-making, the `HealthMonitoring` class computes the health scores and applies the acceptance rules based on the defined criteria. These criteria will be discussed in the next section.

The `LiFCal` class implements the target-free calibration method proposed by Fleith et al. [8]. This class encapsulates the core logic of LiFCal, including the bundle adjustment optimization. It provides methods to run the calibration process and retrieve the estimated camera parameters.

Finally, for results and traceability, the `ResultLogger`, `lifcal_to_json`, and `CalibrationState` classes handle the logging of calibration runs, health evaluations, and decisions. This ensures that all relevant information is recorded in a structured format for later analysis.

3.1.5. Health Monitoring and Decision Metrics

As mentioned in the use case description, a key component of the proposed calibration framework is the health monitoring mechanism. This mechanism evaluates the quality of each recalibration attempt and decides whether to accept or reject the new calibration based on defined metrics. To define these metrics, the followeing formulas are used:

$$\text{HealthScore} = 100 \cdot \Big(1 - \Big(0.6 \cdot E_{\text{reproj}}^2 + 0.4 \cdot E_{\text{rms}}^2\Big)\Big) \tag{9}$$

$$E_{\text{reproj}} = \text{clamp}\Big(\frac{e_{\text{reproj}} - R_{\text{BEST}}}{R_{\text{WORST}} - R_{\text{BEST}}},\ 0,\ 1\Big) \tag{10}$$

$$E_{\text{rms}} = \text{clamp}\Big(\frac{e_{\text{rms}} - S_{\text{BEST}}}{S_{\text{WORST}} - S_{\text{BEST}}},\ 0,\ 1\Big) \tag{11}$$

This health score formulation is used for the initial target-based calibration. It combines normalized reprojection error and RMS error into a single scalar score in the range [0,100], where higher values indicate better calibration quality. Clamp functions ensure that the error terms are bounded between 0 and 1 based on predefined best and worst-case thresholds. The weights are chosen to give slightly higher priority to the reprojection error than to the RMS error. This choice follows common practice in the literature, where reprojection error is treated as a primary indicator of calibration correctness and overall geometric accuracy.

For the LiFCal-based recalibration, a similar health score is computed using LiFCal-specific metrics:

$$\text{HealthScore}_{\text{LiFCal}} = 100 \cdot \Big[1 - \Big(0.6 \cdot E_{\text{std}}^2 + 0.4 \cdot E_{\text{mae}}^2\Big)\Big] \tag{12}$$

$$E_{\text{std}} = \text{clamp}\left(\frac{\sqrt{x_{\text{std}}^2 + y_{\text{std}}^2} - S_{\text{BEST}}}{S_{\text{WORST}} - S_{\text{BEST}}},\ 0,\ 1\right) \tag{13}$$

$$E_{\text{mae}} = \text{clamp}\left(\frac{\sqrt{x_{\text{mae}}^2 + y_{\text{mae}}^2} - R_{\text{BEST}}}{R_{\text{WORST}} - R_{\text{BEST}}},\ 0,\ 1\right) \tag{14}$$

with

$$R_{\text{BEST}} = 0.03, \quad R_{\text{WORST}} = 1.0, \quad S_{\text{BEST}} = 0.0, \quad S_{\text{WORST}} = 50.0$$

This formulation is applied to LiFCal-based recalibration results. It evaluates the stability and consistency of the estimated camera parameters using standard deviation and mean absolute error metrics. The resulting health score enables a direct comparison with the baseline calibration to support automated acceptance or rejection decisions. The initial and recalibration health scores are computed differently due to the different result metrics provided by the respective calibration methods. The specific best and worst-case values are defined also based on calibration performance which we discussed in Calibration Parameters and Error Metrics section.

3.1.6. State Diagram

In order to illustrate the workflow and interactions between the different calibration phases and components, a state diagram is presented in Figure 10. This diagram visualizes the transitions between the initial calibration, periodic recalibration attempts, and health evaluation steps.

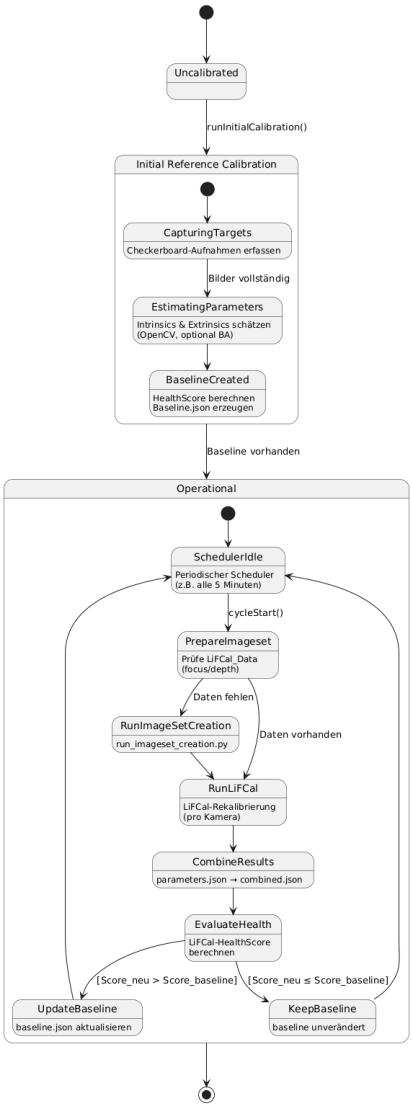

Figure 10: State Diagram

3.2. Materials

In this chapter, the materials and tools used for the implementation and testing of the calibration software are described and presented. Due to the lack of a physical multi-camera light field system, synthetic data generation with Blender was used to create realistic test scenarios. The structure of the image sets, ground truth definitions, and the runtime environment are also discussed.

3.2.1. Runtime Environment and Toolchain

Most parts of the implementation are based on established open-source libraries and tools. The primary programming language used is Python, chosen for its extensive ecosystem supporting computer vision, numerical computation, and scientific prototyping. Core libraries include OpenCV for calibration and image processing, NumPy for numerical operations, and Matplotlib for visualization. The LiFCal implementation provided by Fleith et al. [8] was integrated into the workflow to perform target-free recalibration.

The development and evaluation were conducted on a system running Ubuntu 20.04.6 LTS. A complete list of software dependencies and library versions is documented in the project repository README files to support reproducibility. It should be noted that the LiFCal implementation relies on GPU acceleration and requires a compatible NVIDIA GPU to execute the optimization efficiently. Consequently, LiFCal-based recalibration cannot be performed on systems without suitable GPU support.

3.2.2. Synthetic Data Generation with Blender

To evaluate and test the proposed system, a world scene was simulated using open-source software. This software is Blender [3], which also considers physical properties of light, materials, and camera optics to generate realistic images.

The first step was to create the multi-camera array geometry. As described in the use case, a cross-shaped arrangement with one central camera and twelve surrounding cameras was constructed. Each camera was configured with the same sensor size and a focal length of 50 mm to make the image sets as realistic as possible. The cameras were placed at equal distances from their neighboring cameras, forming a cross shape. Each camera was positioned on a grid to simulate the main structure of the array, as shown in Figure 11.

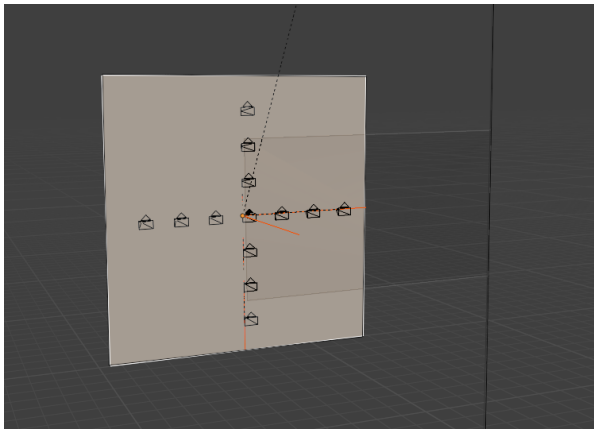

Figure 11: Multi-camera array setup in Blender.

The first scene that needed to be created included a calibration target for the initial calibration. Therefore, a checkerboard pattern was modeled and placed in front of a scene obtained from the Blender community [24]. The checkerboard was positioned in different locations and orientations to simulate a realistic calibration scenario. Due to the multi-camera setup, rendering the images took several minutes per pose, because each camera view had to be rendered individually. After rendering, the images were organized into folders according to the naming conventions required by the calibration scripts. The scene created for the initial calibration was modeled as a simple room, as shown in Figure 12.

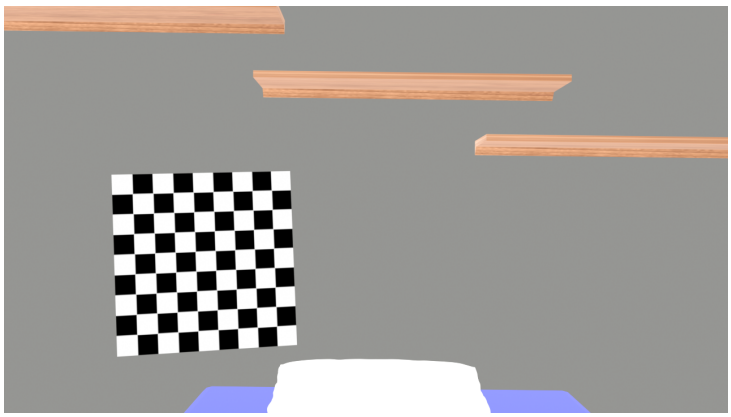

Figure 12: Example of a Blender scene for the initial calibration with a checkerboard.

For the LiFCal-based recalibration, a different scene was created without a calibration target. Since LiFCal relies on feature detection and depth maps, the scene had to be more complex and include a variety of objects and textures. Therefore, another scene from the Blender community was used [22], which represents a modern living room with furniture and decorations. The same camera setup was used, and the scene was rendered from different viewpoints to create the required image sets. An example of the scene used for LiFCal recalibration is shown in Figure 13.

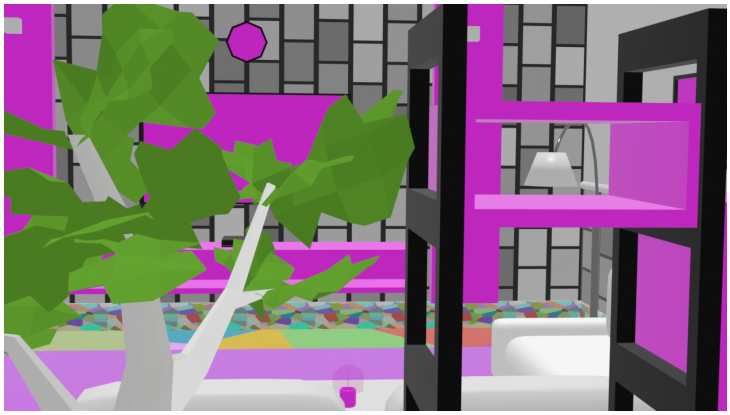

Figure 13: Example of a Blender scene for LiFCal recalibration without a calibration target.

The motivation behind using synthetic data was to maintain full control over the scene configuration and to simulate different calibration scenarios without the need for physical hardware. Another major advantage is the availability of ground truth camera parameters, which is essential for comparing the estimated calibration results with the actual values. This enables a more objective evaluation of calibration accuracy and the effectiveness of the implemented software.

3.2.3. Image Sets and Ground Truth Definition

To evaluate the calibration methods effectively, multiple image sets were created with different numbers of poses and camera views. In addition, for each image set, the multi-camera system was deliberately decalibrated to simulate real-world scenarios in which calibration quality may degrade over time due to mechanical or environmental influences.

The naming conventions for the image sets followed a structured and consistent format to ensure maintainability and usability. Each image set was organized into folders according to the calibration phase (initial calibration or LiFCal-based recalibration). File names encoded both the pose number and the camera identifier, for example `pose_001_Up1.png`.

The ground truth camera parameters were directly extracted from Blender. Since camera positions, orientations, and intrinsic parameters were explicitly defined within the 3D scene, the exact intrinsic and extrinsic parameters were known internally. The ground truth data was stored in JSON format, following the same structure as the calibration results, to enable direct and systematic comparison during evaluation. For each camera, both intrinsic parameters and extrinsic poses were included.

4. Results and Evaluation

4.1. Results of Initial Calibration

4.1.1. Intrinsic Accuracy

4.1.2. Extrinsic Accuracy

4.1.3. Reprojection and RMS Error

4.2. Results of LiFCal-Based Recalibration

4.2.1. Reprojection Error after LiFCal

4.2.2. Health Score Evolution

5. Discussion of the Results

5.1. Interpretation of Initial Calibration Results

5.2. Limitations of LiFCal in the Given Setup

5.3. Comparison between Initial Calibration and LiFCal

5.4. Comparison to state of the Art

5.5. Potential Extensions and Improvements

6. Conclusion and Outlook

6.1. Conclusion

6.2. Outlook

A. Appendix

Acknowledgment

References

[1] Yuriy Anisimov, Gerd Reis, and Didier Stricker. Calibration and auto-refinement for light field cameras. In *Proceedings of the 29th International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision (WSCG)*, pages 253–261, Pilsen, Czech Republic, 2021. Vaclav Skala - UNION Agency. ISBN 978-80-86943-34-3. URL https://www.dfki.de/fileadmin/user_upload/import/11659_wscg-j53.pdf.

[2] CamCalib. The reprojection error. https://www.camcalib.io/post/what-is-the-reprojection-error, 2022. CamCalib blog, accessed 2026-01-21.

[3] Blender Online Community. *Blender - a 3D modelling and rendering package*. Blender Foundation, Stichting Blender Foundation, Amsterdam, 2018. URL http://www.blender.org.

[4] Daniel Cremers. Bundle adjustment & nonlinear optimization. https://cvg.cit.tum.de/_media/teaching/ss2019/mvg2019/material/multiviewgeometry7.pdf, 2019. Vorlesungsunterlage, Technische Universität München.

[5] Elijs Dima, Mårten Sjostrom, and Roger Olsson. Assessment of multi-camera calibration algorithms for two-dimensional camera arrays relative to ground truth position and direction. In *2016 3DTV-Conference: The True Vision - Capture, Transmission and Display of 3D Video (3DTV-CON)*, pages 1–4, 2016. doi: 10.1109/3DTV.2016.7548887. URL https://ieeexplore.ieee.org/document/7548887.

[6] Elsevier. Radial distortion. https://www.sciencedirect.com/topics/computer-science/radial-distortion, 2022. Definition from ScienceDirect Topics: Computer Science.

[7] Elsevier. Lens distortion. https://www.sciencedirect.com/topics/engineering/lens-distortion, 2024.

[8] Aymeric Fleith, Doaa Ahmed, Daniel Cremers, and Niclas Zeller. Lifcal: Online light field camera calibration via bundle adjustment. https://arxiv.org/abs/2408.11682, 2024.

[9] Pengxiang Ge, Huanqing Wang, Yonghong Wang, and Biao Wang. Calibration of ring multicamera system with transparent target for panoramic measurement. *IEEE Sensors Journal*, 22(23):23154–23164, 2022. doi: 10.1109/JSEN.2022.3214080. URL https://ieeexplore.ieee.org/document/9923591.

[10] German Aerospace Center (DLR) – Institute of Robotics and Mechatronics. Light field cameras. DLR Research Expertise: 3D Perception, 2026. URL https://www.dlr.de/en/rm/research/expertise/3d-perception/light-field-cameras. Accessed January 24, 2026.

[11] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, UK, 2nd edition, 2003. ISBN 0521540518. URL http://www.r-5.org/files/books/computers/algo-list/image-processing/vision/Richard_Hartley_Andrew_Zisserman-Multiple_View_Geometry_in_Computer_Vision-EN.pdf.

[12] Jiachun Huang, Shaoli Liu, Jianhua Liu, and Zehua Jian. Camera calibration results: Reprojection error and relative positions. *Scientific Reports*, 2016. URL https://www.researchgate.net/figure/Camera-calibration-results-a-Reprojection-error-and-b-relative-positions-of-the_fig9_301715405. In Fig. 5.

[13] IEEE. Ieee recommended practice for software requirements specifications. *IEEE Std 830-1998*, pages 1–40, 1998. doi: 10.1109/IEEESTD.1998.88286.

[14] Mark Hedley Jones. Calibration checkerboard collection. Online project page, 2025. URL https://markhedleyjones.com/projects/calibration-checkerboard-collection.

[15] Dhiraj Kumar. What is bundle adjustment? Baeldung on Computer Science, 2024. URL https://www.baeldung.com/cs/computer-vision-bundle-adjustment. Last updated March 18, 2024.

[16] Philip K. Lee and Anqi R. Ji. Comparison of lightfield reconstruction methods using undersampled data. Technical Report EE367 Winter 2017 Project Report, Stanford University, Department of Electrical Engineering, 2017. URL https://stanford.edu/class/ee367/Winter2017/lee_ji_ee367_win17_report.pdf.

[17] Fei-Fei Li et al. Camera models. https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf, 2020.

[18] Shih-Hao Lin, En-Tze Chen, Jia-Jun Xiao, and Ching-Yuan Chang. Stereo digital image correlation (3d-dic) for non-contact measurement and refinement of delta robot arm displacement and jerk. *The International Journal of Advanced Manufacturing Technology*, 133(5-6):2809–2822, July 2024. doi: 10.1007/s00170-024-13957-2. URL https://doi.org/10.1007/s00170-024-13957-2.

[19] MathWorks. Camera calibration. https://de.mathworks.com/help/vision/ug/camera-calibration.html, 2024.

[20] OpenCV Contributors. Camera calibration with opencv. https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html, 2024.

[21] OpenCV Developers. Camera calibration and 3d reconstruction — opencv documentation. OpenCV 4.x Documentation, calib3d module, 2025. URL https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html.

[22] rahadianrayhan. Living room 3d model. Free3D online 3D model, 2021. URL https://free3d.com/3d-model/living-room-997877.html. Blender, FBX, and OBJ formats, 20 035 vertices, 18 735 faces, 38 514 triangles.

[23] Carlos Ricolfe-Viala and Antonio-José Sánchez-Salmerón. Lens distortion models evaluation. *Applied Optics*, 49(30):5914–5928, 2010. doi: 10.1364/AO.49.005914. URL https://www.researchgate.net/publication/47510646_Lens_distortion_models_evaluation.

[24] satriacaster. Isometric room 3d model. Free3D online 3D model, 2021. URL https://free3d.com/3d-model/isometric-room-525234.html. Blender (.blend), textures and materials included.

[25] Chaehyeon Song, Dongjae Lee, Jongwoo Lim, and Ayoung Kim. Camera calibration via circular patterns: A comprehensive framework with measurement uncertainty and unbiased projection model. *arXiv preprint arXiv:2506.16842*, 2025. URL https://arxiv.org/abs/2506.16842.

[26] Stanford Computer Graphics Laboratory. The stanford multi-camera array. Project page, Stanford University, 2011. URL https://graphics.stanford.edu/projects/array/.

[27] Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle adjustment – a modern synthesis. In Bill Triggs, Andrew Zisserman, and Richard Szeliski, editors, *Vision Algorithms: Theory and Practice*, volume 1883 of *Lecture Notes in Computer Science*, pages 298–372, London, UK, 2000. Springer-Verlag. doi: 10.1007/3-540-44480-7_21. URL https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Triggs00.pdf.

[28] Zhaomin Xie, Mengxiao Zhao, and Yujia He. Research on depth information acquisition of array camera based on depth map fusion. In *2023 5th International Conference on Frontiers Technology of Information and Computer (ICFTIC)*, pages 759–762, 2023. doi: 10.1109/ICFTIC59930.2023.10456113. URL https://ieeexplore.ieee.org/document/10456113.

[29] Yichao Xu, Kazuki Maeno, Hajime Nagahara, and Rin ichiro Taniguchi. Camera array calibration for light field acquisition. *Frontiers of Computer Science*, 9(5): 691–702, 2015. doi: 10.1007/s11704-015-4237-4. URL https://link.springer.com/article/10.1007/s11704-015-4237-4.

[30] Jungang Yang, Chao Xiao, Pu Wang, Yougan Luo, and Chengjin An. Camera array calibration using a simple checkerboard pattern. In *2019 IEEE International Conference on Signal, Information and Data Processing (ICSIDP)*, pages 1–5, 2019. doi: 10.1109/ICSIDP47821.2019.9172948. URL https://ieeexplore.ieee.org/document/9172948.

[31] Yaning Zhang, Yingqian Wang, Tianhao Wu, Jungang Yang, and Wei An. Fixed relative pose prior for camera array self-calibration. *IEEE Transactions on Circuits and Systems for Video Technology*, 35(1):981–985, 2025. doi: 10.1109/TCSVT.2024.3450706. URL https://ieeexplore.ieee.org/document/10649654.

[32] Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000. doi: 10.1109/34.888718. URL https://ieeexplore.ieee.org/document/888718.

[33] Feng Zhou, Jing Li, Yanran Dai, Lichen Liu, Haidong Qin, Yuqi Jiang, Shikuan Hong, Bo Zhao, and Tao Yang. Time-series fusion-based multicamera self-calibration for free-view video generation in low-texture sports scene. *IEEE Sensors Journal*, 23(3):2956–2969, 2023. doi: 10.1109/JSEN.2022.3230792. URL https://ieeexplore.ieee.org/document/9998530.